

Hierarchical GMS

1st Matthew Wong
University of Washington
Bothell, USA
wongm9@uw.edu

2nd Pranshu Bhardwaj
University of Washington
Bothell, USA
pran@uw.edu

3rd Brennan O'Reilly
University of Washington
Bothell, USA
bren@uw.edu

Abstract—Establishing reliable feature correspondences is a fundamental requirement for many vision tasks, including visual odometry, image alignment, and 3D reconstruction. Classical Grid-Based Motion Statistics (GMS) achieves fast and robust filtering by exploiting local match density, but its performance drops in scenes with local distortions, multiple depth layers, or spatially varying motion. This project introduces a hierarchical extension of GMS that integrates three complementary refinement stages: Hierarchical GMS (HGMS), Local Affine Transformation filtering (LAT), and Multi-Plane motion clustering (MP). HGMS performs multi-resolution statistical filtering, LAT applies cell-level affine verification using RANSAC, and MP separates correspondence sets into motion-consistent clusters. Together, these stages strengthen geometric consistency while retaining real-time feasibility. Experiments on raw ORB matches and the VGG affine dataset show that HGMS consistently improves precision over classical GMS, while LAT and MP provide targeted benefits in locally distorted and multi-planar scenes. The full pipeline demonstrates meaningful improvements in correspondence stability and coherence, highlighting the advantages of combining statistical, local geometric, and multi-model cues within a unified matching framework.

The materials subsection of this document contains the link to the github repo which contains source code and test output.

Index Terms—Feature Matching, GMS, Computer Vision, Local Affine Transformation, Hierarchical Filtering, Multi-Plane

I. INTRODUCTION

Feature correspondence is a core task in computer vision and supports many higher level problems such as 3D reconstruction, visual odometry, object tracking, and image alignment. Modern pipelines often rely on local feature descriptors like SIFT or ORB, but the quality of the final matches depends heavily on filtering methods that remove outliers and enforce geometric consistency. Grid Based Motion Statistics (GMS) is one of the most widely used classical techniques due to its speed and its ability to produce stable correspondences without expensive optimization. GMS organizes matches into grid cells and uses neighborhood statistics to suppress inconsistent motion patterns, which allows it to run in real time on a CPU.

Although GMS is fast and robust, it still struggles in scenes where the underlying geometry varies across the image. Local distortions, depth changes, and multi-plane structure can cause spatially inconsistent motion patterns that reduce the effectiveness of the original grid based approach. These limitations motivated us to explore a hierarchical design that strengthens the filtering process while keeping the computational cost low.

Our project extends the traditional GMS method and adds two additional refinement stages that target different forms of geometric variation. The first stage is Local Affine Transformation (LAT), which performs cell level RANSAC to estimate small local transformations. This helps capture motion patterns that deviate from global assumptions. The second stage is a Multi Plane (MP) refinement that identifies groups of matches that belong to separate planes and filters them using plane specific consistency checks. The third stage is Hierarchical GMS (HGMS), which is an enhanced version of the original GMS algorithm. This stage introduces hierarchical grid filtering to GMS, which allows the algorithm to take feature matches through multiple levels of grid-based feature filtering with different grid resolutions, going from a coarse grid to a smooth grid. By combining these stages into a single hierarchical pipeline, we aim to achieve higher match quality, stronger spatial coherence, and more reliable correspondences for downstream tasks.

II. RELATED WORKS

The problem of establishing feature correspondence is a fundamental task in computer vision, which is underpinning applications such as 3D reconstruction, visual odometry, and image alignment. Work in this domain can be categorized into methods for initial feature matching and subsequent techniques for outlier rejection (geometric verification) [1]. Our work, Multi-stage GMS, is positioned as an advancement in the latter category, specifically designed to enhance robustness in complex, multi-structure environments.

Classic feature matching pipelines rely on local descriptors like SIFT and ORB to generate an initial set of potential correspondences. The conventional approach for filtering these matches involves estimating a single, global geometric model, such as a homography or fundamental matrix, to characterize the motion between the two images. Random Sample Consensus (RANSAC) remains the most dominant method for this task [1]. While RANSAC is robust to a high percentage of outliers, its iterative nature and model fitting step can be computationally demanding, often struggling to maintain real-time performance [1]. More significantly, a single global model is insufficient for complex scenes involving nonrigid motion or multiple, distinct planar structures.

To overcome the speed and computational limitations of RANSAC, a line of research focused on methods that leverage

the inherent smoothness of motion fields. These methods substitute complex geometric optimization with simpler statistical checks. Grid-based Motion Statistics (GMS) is a prominent example [1]. GMS operates on the principle that true matches (inliers) tend to cluster in localized regions, whereas false matches (outliers) are scattered randomly [1]. By organizing correspondences into a grid and using a statistical likelihood test based on the number of neighboring matches, GMS achieves ultra-robust and real-time performance [1]. However, GMS’s reliance on uniform grid cells and an underlying assumption of general motion consistency limits its efficacy in scenes with strong local deformations or multiple independently moving objects.

A major challenge for both global RANSAC and GMS is accurately handling scenes that violate the single-model assumption. Our Local Affine Transformation (LAT) and Multi Plane (MP) stages are motivated by techniques developed to address nonrigid and piecewise-planar correspondence. To handle complex deformations, methods have been developed that focus on local geometric constraints. For instance, Locality Affine-invariant Matching (LAM) is a robust algorithm that models correspondences using a local barycentric coordinate (LBC) system that is invariant to local affine transformations [2]. This approach, suitable for nonrigid matching, extracts good matches by building a mathematical model based on LBCs [2]. Our LAT stage adopts a similar philosophy by locally refining the matches within grid cells using affine models, but integrates it into the statistical GMS framework. For scenes exhibiting multiple planes, recent work has proposed modular, non-deep learning methods to filter and refine sparse correspondences by aggregating them into overlapping clusters corresponding to virtual planes [3]. These methods often use an iterative RANSAC-based approach to discard non-conforming correspondences and refine keypoint positions based on a piecewise-planar approximation [3]. Our MP stage explicitly segments matches into different planar structures and applies specialized checks, echoing the need for a multi-model approach in complex environments.

The use of hierarchical processing is a widely adopted strategy in computer vision to achieve high accuracy and efficiency by combining information from multiple spatial scales. Our Hierarchical GMS (HGMS) stage, which performs grid filtering at multiple resolutions (coarse to fine), is consistent with this paradigm. Recent advancements in feature matching, particularly those using Transformer-based architectures, have explicitly focused on integrating information across different scales. For example, some methods employ a multi-layer processing and coarse-grained filtering network to address the limitations of models that neglect the integration of global, local, and multi-scale features [4]. Furthermore, detector-free matching techniques have leveraged coarse and fine features through modules like local windows aggregation to improve matching performance in weak texture regions [5]. These efforts demonstrate the clear benefit of having multiple stages or scales of analysis. Our HGMS stage applies this hierarchical concept directly to the GMS framework, using

a multi-resolution grid to capture motion coherence ranging from coarse-grained global patterns to smooth-grained local details.

By integrating the principles of local affine correction (LAT), multi-structure awareness (MP), and multi-resolution refinement (HGMS) into the high-speed GMS framework, our Multi-stage GMS offers an efficient and robust solution for feature correspondence in geometrically complex scenes, setting it apart from purely global, purely statistical, or purely local approaches.

III. METHODOLOGY

Our goal was to design, implement, and evaluate a full hierarchical extension of the Grid Based Motion Statistics (GMS) pipeline. The resulting system contains three major stages: Hierarchical GMS (HGMS), Local Affine Transformation refinement (LATStage), and Multi Plane filtering (MPStage). Each stage targets a different weakness of classical GMS and contributes incrementally to match refinement. This section describes the design decisions, algorithms, and implementation details of each stage.

A. Pipeline Overview

The foundation of a multi-stage processing approach to filter matching is the image processing or feature extraction pipeline. A typical pipeline for feature matching may consist of multiple stages that include image processing, feature extraction and detection, matching, and finding best-matching pixel between source and target images [8]. Our hierarchical multi-stage processing approach is different from the typical pipeline in that our approach focuses on the filtering of outliers utilizing multiple stages. Each stage contains a specific algorithm inspired by GMS for filtering of feature match outliers. The pipeline stages can be registered in any order and only one instance of a specific stage can be registered within the pipeline. The pipeline operates in two modes: filter or aggregate. When the pipeline is configured for the filter mode then the output from one stage serves as the input into the next stage. Meaning if a stage that is aggressive in match filtering is registered first in the pipeline, then any downstream stages may have limited inputs to filter. If the pipeline is configured in aggregate mode, then the order of how stages are added to the pipeline do not make a difference. Each stage will receive the same feature match inputs when the pipeline is in aggregate mode. The filtered feature match output from each stage is aggregated together and duplicate matches are removed. The duplicates are determined by creating a compound key of the source keypoint indexes and the target keypoint indexes and keeping only unique combinations of the compound key.

The work in the multi-stage pipeline mainly takes place in the three stages with additional aggregation and removal of duplicate records taking place in the pipeline itself. The pipeline also records run metrics including matches retained from each stage and makes those accessible to the caller. The pipeline utilizes a similar method signature to the GMS match method and is meant as a drop-in replacement for GMS.

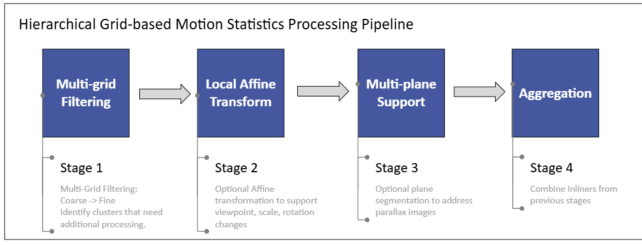


Fig. 1: Filter Pipeline with all stages

The three stages included with this implementation are the following:

- 1) HGMS Stage: multi-resolution grid filtering
- 2) LAT Stage: local affine RANSAC refinement inside grid cells
- 3) MP Stage: motion vector clustering and plane specific filtering

B. HGMS Stage

The first stage of our Multi-stage GMS pipeline is the Hierarchical Grid-based Motion Statistics (HGMS) refinement. This stage extends the original GMS algorithm [1] by introducing a mechanism for cascaded statistical filtering across multiple spatial resolutions, thereby increasing the robustness and adaptive capacity of the matcher without sacrificing its real-time performance.

The HGMS module is structurally distinct from the original GMS [1] in two key aspects:

1. Hierarchical Grid Resolutions: Instead of relying on a single fixed grid resolution, HGMS defines and processes a sequence of grid sizes. The matcher initializes internal structures to support these multiple grid sizes, allowing it to progressively filter matches from a coarse level to a fine level. The final grid sizes were chosen empirically after extensive experimentation with various combinations to optimize the trade-off between computational cost and matching accuracy. The three resolutions employed in our implementation are 15×15 (coarse), 20×20 (medium), and 25×25 (fine).

2. Progressive Inlier Filtering: The filtering process in HGMS is performed progressively through each grid stage. The output inlier mask from a coarser stage serves as the constrained input for the subsequent, finer stage. Specifically, the `getInlierMask` function iterates through the defined grid sizes, and for each size, it runs the core GMS logic (`runSingleStage`) using only the matches that were confirmed as inliers in all previous stages. This progressive update of the inlier mask enables a more robust filtering process by ensuring that only matches that exhibit statistical coherence across multiple scales are retained in the final set.

By adopting this hierarchical, multi-resolution approach, HGMS ensures that large-scale motion patterns are first verified by the coarse grid, and then finer details and local variations are validated by subsequent, higher-resolution grids. Furthermore, the handling of scale is preserved from the

original GMS by adjusting the right image grid size based on predefined scale ratios at each hierarchical level.

Algorithm 1: Hierarchical Grid-based GMS (HGMS)

Input: Keypoints $\mathbf{K}_1, \mathbf{K}_2$;
Image sizes S_1, S_2 ;
Initial matches $\mathcal{M}_{all}$;
Threshold factor λ ;
Flags `withRotation`, `withScale`.
Output: Filtered inlier matches $\mathcal{M}_{HGMS}$

Initialization:
1: Normalize keypoints: $\mathbf{p}_1 \leftarrow \text{normalize}(\mathbf{K}_1, S_1)$,
 $\mathbf{p}_2 \leftarrow \text{normalize}(\mathbf{K}_2, S_2)$;
2: Convert matches to pairs $\text{mvMatches} \leftarrow (\text{queryIdx}, \text{trainIdx})$;
3: Initialize inlier mask $\text{mask}[i] \leftarrow \text{true}$ for all i ;
4: Define hierarchical grid sizes: $\{G_1, G_2, G_3\}$ (Coarse $\rightarrow$ Fine);

Step 1: Hierarchical Grid Filtering
5: **foreach** grid size $G \in \{G_1, G_2, G_3\}$ **do**
6: Initialize left grid $G_L \leftarrow G$ and right grid G_R based on scale ratios;
7: Initialize neighbors for left and right grids;
8: Store input mask from previous stage: $\text{stageMask} \leftarrow \text{mask}$;
9: Initialize best mask for this stage: $\text{bestMask} \leftarrow \text{stageMask}$;
10: **for** scale = 0 to (withScale ? 4 : 0) **do**
11: Set current right grid G_R according to scale;
12: **for** rotationType = 1 to (withRotation ? 8 : 1) **do**
13: Reset working mask: $\text{mask} \leftarrow \text{stageMask}$;
14: Compute motion statistics for all matches still marked as inliers;
15: Assign matches to left-right grid cell pairs;
16: Verify cell pairs using rotation pattern and threshold factor;
17: Update temporary inliers for this shift;
18: Update mask and track number of inliers;
19: If current inliers i best in stage, update bestMask ;
20: **end**
21: Set output mask for this stage: $\text{mask} \leftarrow \text{bestMask}$;
end

Step 2: Collect final inliers
22: Initialize $\mathcal{M}_{HGMS} \leftarrow \emptyset$;
23: **for** $i = 1$ to $|\mathcal{M}_{all}|$ **do**
24: **if** $\text{mask}[i] = \text{true}$ **then**
25: Append $\mathcal{M}_{all}[i]$ to $\mathcal{M}_{HGMS}$;
end
end

return $\mathcal{M}_{HGMS}$;

C. LAT Stage

The second stage, local affine transform (LAT) stage, is designed to handle geometric variations that are not fully captured by HGMS. The GMS paper groups match neighborhoods using a smooth lateral motion assumption which can limit in-plane rotational invariance[1]. Another key assumption of the GMS paper is that motion is smooth over a large area. However local distortions such as parallax or small perspective changes often cause feature motion that is not well described by uniform neighborhood statistics. To address the challenges, LAT performs local affine refinement using a fixed 20×20 grid over the image. RANSAC is selectively used to predict how a neighborhood may transform and then compares the prediction against the result to determine if the match is a true match. The gridding/scoring approach adopted by the LAT stage is very similar to GMS with the exception that matches in a specific area within the source image are not expected to be in the same area within the target image.

Images are divided into 20×20 non-overlapping cells and scores are calculated based on numbers of matches in a 3×3 neighborhood region similar to GMS. For each

match, LAT computes a source cell index in the first image and a destination cell index in the second image. These cell assignments are used to populate a 400 x 400 counting matrix that stores the number of matches that map from each source cell to each destination cell. The counting matrix is used as an optimization similar to GMS so that the scoring becomes a counting problem versus a complex algorithm. The LAT stage looks for clusters of matches that exceed the RANSAC threshold and then using RANSAC, generates an affine transformation matrix. RANSAC is selectively used on match clusters that exceed a threshold for performance. The affine transformation matrix is applied to the center of the source cell to predict where the corresponding location should lie in the second image. If the actual target location matches the predicted target within the threshold, then the match is considered an inlier.

Next, LAT evaluates the predicted location using a 3 x 3 neighborhood in the counting matrix. This neighborhood score indicates how well the affine prediction agrees with the observed match distribution. Cells with high neighborhood scores are considered geometrically consistent. Finally, matches belonging to those cells are retained if the cell's score exceeds a threshold based on the user supplied threshold factor. This produces a refined set of locally verified correspondences.

D. MP Stage

The third stage is designed to improve performance in scenes that contain multiple depth layers or independent motions. Even with local affine refinement, correspondences from different planes may interfere with one another if they share overlapping grid regions. The MP stage resolves this by grouping matches according to their motion vectors.

For each match, the motion vector is defined as the difference between the matched keypoint coordinates in the two images. The method clusters these motion vectors using k means with two clusters. Each cluster is treated as a separate motion plane. For each plane, the stage computes the mean motion vector and the variance of motion vectors within the cluster. A consistency threshold is then computed using a statistical model that scales with the user supplied threshold factor.

Matches are retained if their motion vector lies within the allowable deviation from the plane's mean vector. This operation serves as a lightweight, GMS inspired filtering process that operates independently for each plane. After processing both clusters, the inliers from all planes are merged to form the final refined match set. If no matches survive the filtering step, the method falls back to the original input matches to avoid degeneracy.

E. Benchmark Evaluation Pipeline

To evaluate the accuracy of the filtering stages, the system includes benchmarking tools for the *VGG Affine Covariant Regions Datasets* [25]. The *VGGBenchmarkRunner* class loads each image pair, computes ORB keypoints, performs brute force matching, and then evaluates baseline performance,

Algorithm 2: Local Affine Transformation Stage (LAT-GMS)

Input: Keypoints $\mathbf{K}_1, \mathbf{K}_2$;
Image sizes S_1, S_2 ;
Raw matches $\mathcal{M}_{\text{all}}$;
Threshold factor λ .
Output: Refined inlier matches $\mathcal{M}_{\text{LAT}}$

Initialization:
1: Let grid size be 20×20 , number of cells $C = 400$.;
2: Initialize counting matrix $M \in \mathbb{Z}^{C \times C}$ with zeros.;
3: Initialize lists $\mathcal{L}_i$ for all source cells $i \in [1..C]$.;

Step 1-2: Discretize matches into grid cells and count transitions
4: **foreach** match $m \in \mathcal{M}_{\text{all}}$ **do**
5: Extract points $p_1 \leftarrow \mathbf{K}_1[m.\text{queryIdx}], p_2 \leftarrow \mathbf{K}_2[m.\text{trainIdx}]$;
6: Compute source cell $c_s \leftarrow \text{cellIndex}(p_1, S_1)$;
7: Compute target cell $c_t \leftarrow \text{cellIndex}(p_2, S_2)$;
8: **if** c_s, c_t **valid** **then**
9: $M(c_s, c_t) \leftarrow M(c_s, c_t) + 1$;
10: Append match index to $\mathcal{L}_{c_s}$;
11: **end**
end

Step 3: Estimate local affine models per source cell
11: **for** $i = 1$ **to** C **do**
12: **if** $|\mathcal{L}_i| \geq T_{\text{RANSAC}}$ **then**
13: Build point sets $P_1 = \{p_1\}, P_2 = \{p_2\}$ from matches in $\mathcal{L}_i$;
14: Estimate affine transform
15: $A_i \leftarrow \text{estimateAffine2D}(P_1, P_2)$ using RANSAC;
16: **end**
end

Step 4: Predict target cell via affine transform
15: Initialize score vector $\sigma[1..C] \leftarrow 0$.;
16: **for** $i = 1$ **to** C **do**
17: **if** A_i **exists** **then**
18: Compute source cell center $q_i \leftarrow \text{cellCenter}(i)$;
19: Apply affine: $\hat{q}_i \leftarrow A_i(q_i)$;
20: Compute predicted target cell $\hat{c}_t \leftarrow \text{cellIndex}(\hat{q}_i, S_2)$;
21: Sum 3x3 neighborhood vote: $\sigma[i] \leftarrow \sum_{u \in \mathcal{N}(\hat{c}_t)} M(i, u)$;
22: **end**
end

Step 5: Select final inliers
22: Initialize $\mathcal{M}_{\text{LAT}} \leftarrow \emptyset$;
23: **for** $i = 1$ **to** C **do**
24: **if** $\sigma[i] \geq \lambda \cdot T_{\text{final}}$ **then**
25: Append all matches in $\mathcal{L}_i$ to $\mathcal{M}_{\text{LAT}}$;
26: **end**
end

return $\mathcal{M}_{\text{LAT}}$;

GMS performance, and HGMS performance. The evaluation uses the *HomographyEvaluator* class, which applies the ground truth homography to the source keypoints and compares the transformed coordinates with the observed matches. If the difference in distance between the observed matches and the predicted matches is less than the threshold (3px) then the match is considered an inlier match. Considering detected inlier matches as true positives, matches that fall outside of the threshold as false matches, and keypoints in ground truth not matched as false negatives we are able to compute the Precision, recall, and F-score [6]. The process for computing the Precision, recall and F-score is very similar to the process utilized by the original GMS [1] paper and also the paper by K. Mikolajczyk, et al. [6]. These metrics provide quantitative evidence of the improvements introduced by the multi-stage filtering approach and also for comparison of performance between no filtering, GMS filtering, and Hierarchical GMS (multi-stage) filtering approaches.

Algorithm 3: Multi-plane Stage (MPStage)

Input: Keypoints $\mathbf{K}_1, \mathbf{K}_2$;
Image sizes S_1, S_2 ;
Raw matches $\mathcal{M}_{\text{all}}$;
Threshold factor λ .
Output: Refined inlier matches $\mathcal{M}_{\text{MP}}$

Initialization:
1: Initialize $\mathcal{M}_{\text{MP}} \leftarrow \emptyset$;
2: If $\mathcal{M}_{\text{all}}$ or $\mathbf{K}_1$ or $\mathbf{K}_2$ is empty, **return**;

Step 1: Compute motion vectors and cluster into planes
3: Compute motion vectors $\mathbf{v}_i \leftarrow \mathbf{K}_2[m.\text{trainIdx}] - \mathbf{K}_1[m.\text{queryIdx}]$ for all $m \in \mathcal{M}_{\text{all}}$;
4: If $|\mathbf{v}| < 3$, set $\mathcal{M}_{\text{MP}} \leftarrow \mathcal{M}_{\text{all}}$ and **return**;
5: Set number of planes $K \leftarrow 2$;
6: Cluster motion vectors into K planes $\{L_i\}_{i=1}^{|\mathcal{M}_{\text{all}}|}$ using k-means;
7: Count non-empty planes nonZeroPlanes ;
8: If $\text{nonZeroPlanes} \leq 1$, set $\mathcal{M}_{\text{MP}} \leftarrow \mathcal{M}_{\text{all}}$ and **return**;

Step 2: Filter matches within each plane
9: Initialize plane-wise match lists $\mathcal{P}_1, \dots, \mathcal{P}_K \leftarrow \emptyset$;
10: **for** $i = 1$ **to** $|\mathcal{M}_{\text{all}}|$ **do**
 11: Assign match m_i to plane $\mathcal{P}_{L_i}$;
end
12: Initialize aggregated inliers $\mathcal{I} \leftarrow \emptyset$;
13: **for** $k = 1$ **to** K **do**
 14: **if** $\mathcal{P}_k$ **not empty** **then**
 15: Compute plane motion mean μ_k and std deviation σ_k ;
 16: Compute threshold $T_k = \lambda \cdot \sigma_k$;
 17: **foreach** match $m \in \mathcal{P}_k$ **do**
 18: If motion of m within T_k from μ_k , add m to $\mathcal{I}$;
 end
 end
end
19: **if** $\mathcal{I}$ **not empty** **then**
 20: $\mathcal{M}_{\text{MP}} \leftarrow \mathcal{I}$;
end
21: **else**
 22: $\mathcal{M}_{\text{MP}} \leftarrow \mathcal{M}_{\text{all}}$;
end
return $\mathcal{M}_{\text{MP}}$;

IV. EXPERIMENTS / RESULTS

This section presents the datasets utilized for testing purposes and for generating metrics to compare against GMS performance. The qualitative and quantitative results of the multi-stage feature correspondence pipeline in both filter mode and aggregate mode are provided in subsequent sections of this document. We evaluate each stage of the system and compare the outputs against classical GMS under several configurations.

All tests utilized the OpenCV Orb feature detector with 10,000 features and the OpenCV bruteforce matcher with norm hamming which is recommended in OpenCV for binary string based descriptors like ORB[7]. 10,000 features was selected as the GMS paper found 10,000 features to be empirically the "maximum number permissible for real-time performance" [1].

A. Datasets

We evaluate multi-stage pipeline processing performance using the *VGG Affine Covariant Regions Datasets* [25] data set. The VGG datasets were specifically selected as they are utilized in the GMS [1] paper for performance testing. The VGG dataset contains 8 sets of images of different subjects of which 7 sets were used for experimentation and generating

comparison metrics. Each set contains 6 images and 6 related homography files that serve as the ground truth. The image sets provide viewpoint changes, zoom+rotation changes, and blur which are all computer vision challenge areas that our multi-stage processing pipeline is focused on addressing.

B. Results

We analyze Hierarchical GMS multi-stage approach against baseline of ORB with bruteforce matching and GMS.

Precision, Recall, F-score:

We compare our Hierarchical GMS approach against GMS and the baseline of no match filtering. While the GMS paper utilizes multiple methods for performance evaluation, the main method for comparing performance of GMS using the VGG affine covariant dataset was to calculate the Precision, Recall, F-score for comparison of GMS against ratio-test.

1) *No Feature Match Filter Baseline:* To illustrate the difficulty of the correspondence problem and motivate the need for filtering, we first show the raw ORB matches before any refinement. (See Fig. 2)



Fig. 2: Raw ORB correspondences before filtering. The match set contains many outliers and spatially inconsistent matches.

2) *GMS Results:* We evaluate the classical Grid Based Motion Statistics algorithm under three standard configurations. These settings represent the baseline performance for motion statistics filtering and serve as a direct comparison point for the hierarchical extension. (See Fig. 3)

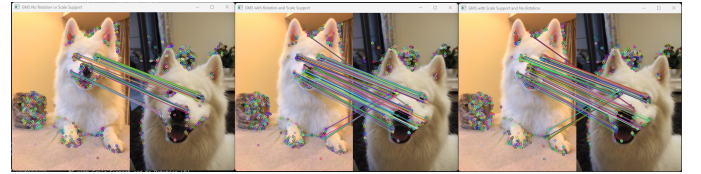


Fig. 3: GMS without rotation or scale support (left), GMS with rotation and scale support (middle), GMS with scale support only (right)

3) *Hierarchical GMS Results:* Next, we evaluate the Hierarchical GMS stage. The multi resolution grid structure provides stronger neighborhood support and more stable filtering across different scales. (See Fig. 4). HGMS stage supports additional rotation and scale flags that can improve the filtering of features matches (see Fig. 4).

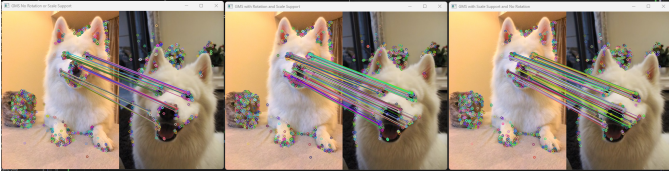


Fig. 4: HGMS without rotation or scale support (left), HGMS with rotation and scale support (middle), HGMS with scale support only (right)

4) *Full Pipeline Output*: Finally, we apply all three stages sequentially (HGMS, LAT, MP) using filter mode and with HGMS scale support flag enabled. The combined system produces the most stable and coherent set of correspondences. (See Fig. 5)

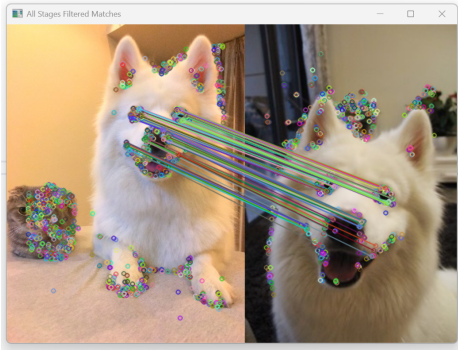


Fig. 5: Output with HGMS, LATStage, and MPStage

5) *Additional Examples*: The following examples demonstrate the robustness of HGMS with scale and object transformation changes. The results for HGMS are in the top of the image set and GMS are in the bottom of the image set.

Fig. 7 HGMS was able to retain true matches that were filtered out by GMS (right hand of spoiler for example and car emblem on hood of car.) Fig 8. HGMS retained matches that were filtered out by GMS as the subject was rotated.

C. Quantitative Evaluation on the VGG Affine Dataset

The precision, recall and F-score are computed using the HomographyEvaluator for each image pair and VGGBenchmarkRunner. The pipeline is tested in both filter mode (where output from one stage serves as input into the

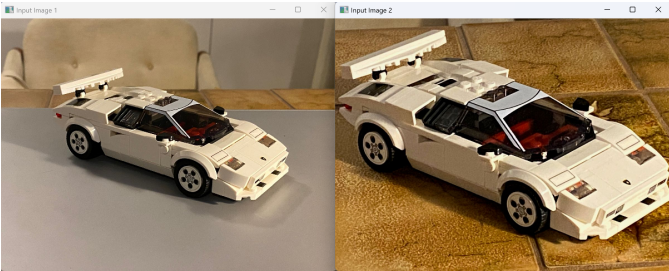


Fig. 6: Image set with subject at difference scales.

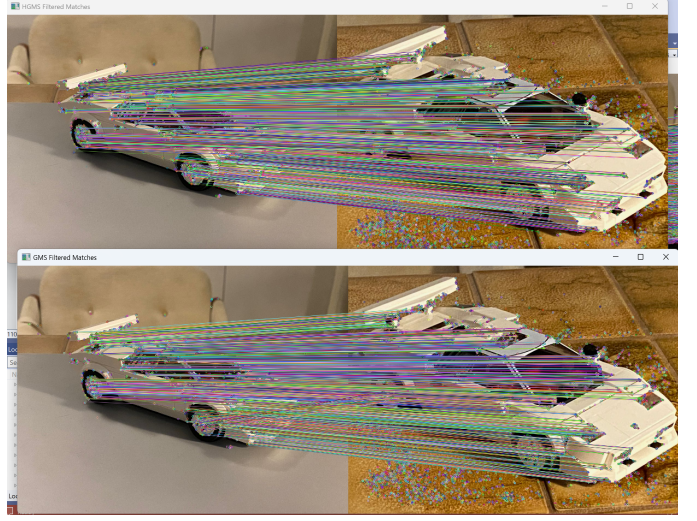


Fig. 7: HGMS versus GMS (HGMS top image and GMS bottom image)

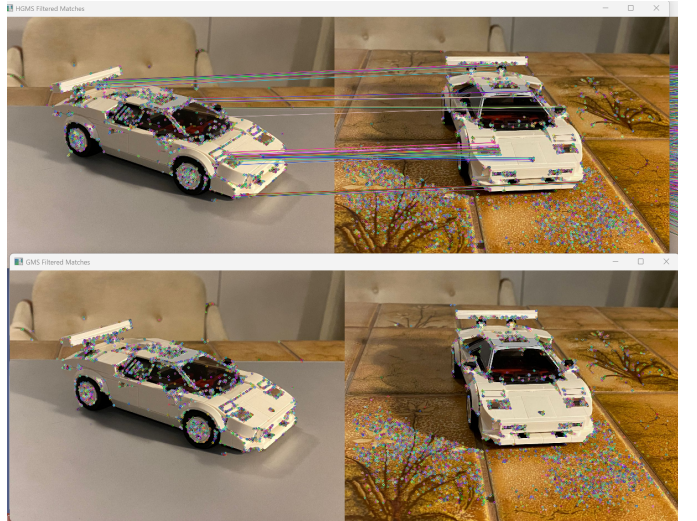


Fig. 8: HGMS versus GMS Perspective Change (HGMS top image and GMS bottom image)

next stage) and aggregation mode (where each stage receives the same set of input and the resulting matches are aggregated together and duplicate matches are removed). The precision, recall, and F-score metrics are computed for the baseline (no filtering), GMS, and Hierarchical multi-stage approach with all stages and for each individual stage.

The data presented in Table I and Table II show the precision, recall, F-score for each of the 7 VGG imagesets. Table I captures the baseline performance without any filtering of feature matches and then captures the performance of the GMS algorithm, the hierarchical multi-stage pipeline performance for each stage and the performance across all stages using filter mode. Table II captures the base performance without any filtering of feature matures and the performance of the hierarchical multi-stage pipeline with all stages in aggregate

mode.

The blue lines represent the baseline metrics, the red lines represent the GMS metrics, and the green lines represent the Hierarchical multi-stage metrics. The long dashed lines represent the recall (the number of matches found that are true matches), the short dashed lines represent the precision (the number of true matches found), and the F-score represents the balance between precision and recall for the algorithm. Higher numbers towards 1 are better scores.

1) *Pipeline Processing Filter Mode*: The following set of figures capture the performance of the multi-stage pipeline in filter mode for all stages and then each individual stage (HGMS, LAT, MP). The performance of all stages versus the performance of each individual stage was captured for comparison purposes and to determine filtering impact of each stage. (See Fig. 9, 10, 11, 12)

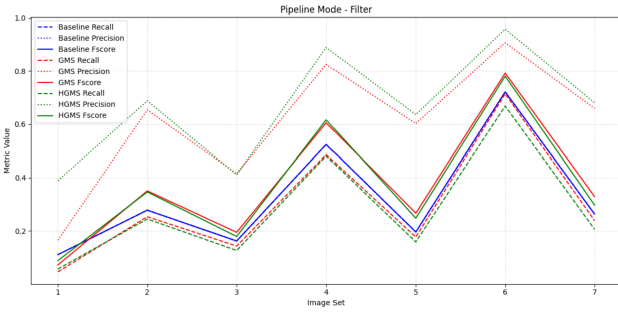


Fig. 9: Filter Pipeline with all stages

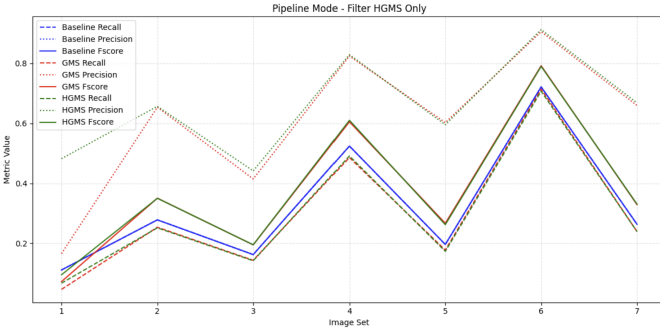


Fig. 10: Filter Pipeline with only HGMS

2) *Pipeline Processing Aggregation Mode*: In addition to capturing the performance of the multi-stage pipeline in filter mode the following figure also captures the performance of the multi-stage pipeline in aggregation mode. Performance of the multi-stage pipeline in aggregation mode was captured to compare against filter mode and to determine how the aggregation of filtered matches impacts overall performance. (See Fig. 13)

D. Qualitative Evaluation on the VGG Affine Dataset

Table I, II provide the quantitative results (Recall, Precision, F-score) from the *VGG Affine Covariant Regions Datasets* [25]. Table III provides the qualitative interpretation of the results from the same dataset.

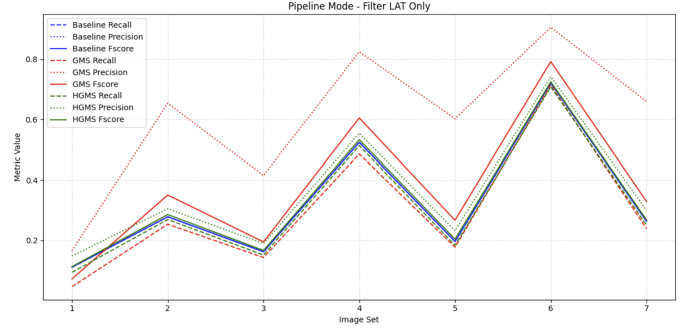


Fig. 11: Filter Pipeline with only LAT

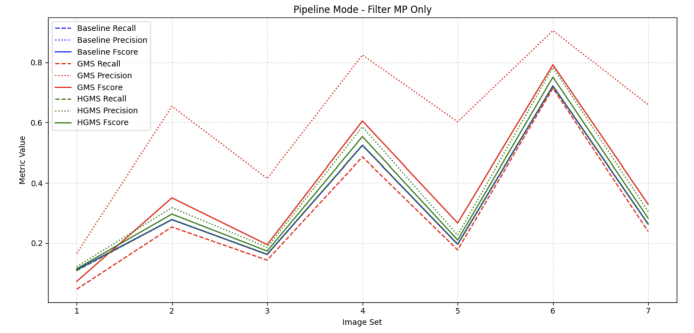


Fig. 12: Filter Pipeline with only MP

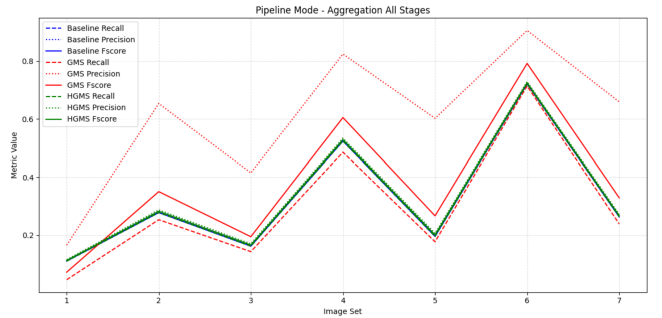


Fig. 13: Aggregate Pipeline HGMS, LAT, MP

TABLE I: Filter Mode Results: All Stages vs Individual Stages (All Image Sets)

Set	Baseline	GMS	HGMS	LAT	MP	All Stages
1	P:0.11 R:0.11 F:0.11	P:0.17 R:0.05 F:0.07	P:0.48 R:0.07 F:0.09	P:0.15 R:0.09 F:0.11	P:0.12 R:0.11 F:0.11	P:0.39 R:0.06 F:0.09
2	P:0.28 R:0.28 F:0.28	P:0.65 R:0.25 F:0.35	P:0.66 R:0.25 F:0.35	P:0.30 R:0.27 F:0.29	P:0.32 R:0.28 F:0.30	P:0.69 R:0.25 F:0.35
3	P:0.16 R:0.16 F:0.16	P:0.41 R:0.14 F:0.20	P:0.44 R:0.14 F:0.19	P:0.19 R:0.15 F:0.17	P:0.19 R:0.16 F:0.17	P:0.41 R:0.13 F:0.18
4	P:0.52 R:0.52 F:0.52	P:0.82 R:0.49 F:0.61	P:0.83 R:0.49 F:0.61	P:0.55 R:0.51 F:0.53	P:0.59 R:0.52 F:0.55	P:0.89 R:0.48 F:0.62
5	P:0.20 R:0.20 F:0.20	P:0.60 R:0.18 F:0.27	P:0.60 R:0.17 F:0.26	P:0.23 R:0.18 F:0.20	P:0.23 R:0.20 F:0.21	P:0.64 R:0.16 F:0.25
6	P:0.72 R:0.72 F:0.72	P:0.91 R:0.71 F:0.79	P:0.91 R:0.71 F:0.79	P:0.74 R:0.71 F:0.72	P:0.78 R:0.72 F:0.75	P:0.96 R:0.67 F:0.78
7	P:0.26 R:0.26 F:0.26	P:0.66 R:0.24 F:0.33	P:0.67 R:0.24 F:0.33	P:0.30 R:0.25 F:0.27	P:0.31 R:0.26 F:0.28	P:0.68 R:0.21 F:0.30

TABLE II: Aggregation Mode Results: All Stages (All Image Sets)

Set	Baseline	GMS	HGMS
1	P:0.11 R:0.11 F:0.11	P:0.17 R:0.05 F:0.07	P:0.12 R:0.11 F:0.11
2	P:0.28 R:0.28 F:0.28	P:0.65 R:0.25 F:0.35	P:0.29 R:0.28 F:0.28
3	P:0.16 R:0.16 F:0.16	P:0.41 R:0.14 F:0.19	P:0.17 R:0.16 F:0.17
4	P:0.52 R:0.52 F:0.52	P:0.82 R:0.49 F:0.61	P:0.53 R:0.52 F:0.53
5	P:0.20 R:0.20 F:0.20	P:0.60 R:0.18 F:0.27	P:0.21 R:0.20 F:0.20
6	P:0.72 R:0.72 F:0.72	P:0.91 R:0.71 F:0.79	P:0.73 R:0.72 F:0.73
7	P:0.26 R:0.26 F:0.26	P:0.66 R:0.24 F:0.33	P:0.27 R:0.26 F:0.27

TABLE III: Qualitative Comparison of HGMS Pipeline Configurations in Filter Mode

Configuration	Overall Behavior	Precision Trend	Recall Trend	F-score Trend	Interpretation / Key Insight
All Stages (HGMS + LAT + MP)	Strong precision but recall often at or slightly below GMS; moderate F-score improvements on structured scenes	High precision; exceeds GMS on most sets	Recall slightly below GMS except in Set 4	Close to GMS; sometimes slightly worse	Additional stages over-filter weak matches, reducing recall. Helps on structured scenes with strong geometric cues.
HGMS Only	Most balanced and consistent variant; generally matches or exceeds GMS	High precision, typically $\geq$ GMS	Recall similar to GMS	Often best or tied for best F-score	HGMS alone provides the best balance. Avoids filtering errors introduced by LAT and MP; strongest standalone model.
LAT Only	High recall but significantly reduced precision	Lowest precision among all variants	Highest recall, often close to baseline	Lower F-score due to precision	LAT accepts too many weak matches. Best suited for tasks prioritizing completeness rather than accuracy.
MP Only	Similar to LAT but slightly more conservative; boosts recall with smaller precision loss	Low precision, but higher than LAT	High recall, slightly below LAT but above GMS on many sets	Lower F-score than HGMS/GMS	MP retains more true matches than LAT but introduces many false positives. Good for moderate recall improvements.

V. CONCLUSION

This work presented a hierarchical extension of the classical Grid-Based Motion Statistics framework that improves correspondence reliability in scenes that challenge traditional motion consistency assumptions. By integrating three complementary refinement stages, Hierarchical GMS, Local Affine Transformation filtering, and Multi-Plane clustering, the system combines multi-scale statistics, local geometric modeling, and motion segmentation within a unified pipeline.

Our experiments show that the HGMS stage provides the strongest standalone improvements, offering higher precision and more stable match distributions than classical GMS across a variety of transformations. LAT and MP contribute targeted benefits in specific scenarios. LAT increases robustness in the presence of local distortions by enforcing affine consistency, and MP preserves valid correspondences in scenes that contain multiple depth layers or independently moving structures. Although combining all stages can reduce recall in some cases because of aggressive filtering, the full pipeline still produces highly coherent and visually stable match sets, especially in structured or planar environments.

Quantitative evaluation on the VGG affine dataset confirms that hierarchical and local refinements can meaningfully raise correspondence quality while keeping computational cost manageable. These results highlight the value of combining statistical, geometric, and clustering-based techniques to address the limitations of purely global or purely local models.

The computational performance of the hierarchical multi-stage approach is fast enough for live video. Once concern raised by the team was the impact of adding additional filtering stages to processing video. The demo and our findings confirm that while there is additional computations being performed that the overall performance allows for filtering of feature matches in live video.

Future work includes exploring adaptive thresholding methods that reduce over-filtering, integrating learned feature descriptors to replace ORB, and extending the pipeline to support temporal tracking in video sequences. Additional improvements may involve confidence modeling or soft inlier scoring, which could provide smoother transitions between stages.

1) *Lessons Learned:* Through the design and development our team learned many lessons related to feature matching and filtering of matches to true inlier matches. Filtering feature matches to true matches is a challenging domain and a technique that may work for one set of images may not be robust for a different set of images. Factors like scale, rotation, inconsistent image motion can all factor into the challenge of filtering feature matches. We learned how algorithm performance highly depends on the image datasets being evaluated and the thresholds selected for determining true feature matches. While development the code base we specifically learned the following:

- Loading a color image as grayscale can impact the feature match filtering negatively.

- The robustness of an algorithm may mask bugs. In this instance two image size parameters are passed into the HGMS algorithm. The same size parameter was initially being passed in (bug) and filtered results were still being returned though at a lower performance. Due to team members seeing different results and stepping through the code we realized a bug in the size parameter being passed into the HGMS match algorithm.
- Establishing how to measure ground truth varies from test set of images to test set of images. The variability of how ground truth is established and evaluated make establishing robustness claims challenging as well for a specific algorithm.
- Importance of testing code with multiple differing image datasets and on different computers/platforms to evaluate results.
- The benefit of reviewing design concept/ideas with other students in the class to get feedback and considerations to think about with the concept and implementation.

2) *Materials:* The source code, demo video, demo files, test results are allocated at the following github repo location: <https://github.com/m4tthewwong/Hierarchical-GMS>

VI. TEAM CONTRIBUTIONS

Here we specify which team members worked on which parts.

Concept / Design:

- Matthew Wong - Multiplane Feature Filtering
- Pranshu Bhardwaj - Hierarchical GMS
- Brennan O'Reilly - Local Affine Transform
- Team - pipeline and multi-stage approach

Code / Test Results:

- Matthew Wong - MPStage, HGMSDemo, Unit Testing
- Pranshu Bhardwaj - HGSMMatcher, HGMSStage, HGMSDemo, Unit Testing
- Brennan O'Reilly - ExecutionMetrics, HomographyEvaluator, Unit Testing VGGBenchmarkRunner, Pipeline, ProcessingStage, LATStage, HGMSDemo

Paper:

- Matthew Wong - Abstract, Introduction, Methodology, HGMS Stage, LAT Stage, MP Stage, Experiments / Results, Conclusion
- Pranshu Bhardwaj - Related Works/Literature Review
- Brennan O'Reilly - Methodology, HGMS Stage, LAT Stage, MP Stage, Experiments / Results, Charts, Tables

REFERENCES

- [1] J. Bian et al., "GMS: Grid-Based Motion Statistics for Fast, Ultra-Robust Feature Correspondence," CVPR 2017.
- [2] J. Li, Q. Hu, and M. Ai, "LAM: Locality affine-invariant feature matching," *ISPRS Journal of Photogrammetry and Remote Sensing*, vol. 154, pp. 28–40, 2019.
- [3] F. Bellavia, Z. Zhao, L. Morelli, and F. Remondino, "Image matching filtering and refinement by planes and beyond," *arXiv preprint arXiv:2411.09484*, 2024.
- [4] Y. Guo, W. Li, and P. Zhai, "A multi-layer processing and coarse filtering network for accurate feature matching," *Pattern Recognition*, vol. 172, p. 112464, 2026.

- [5] Y. Guo, W. Li, P. Zhai, and L. Wu, "Feature matching based on local windows aggregation," *iScience*, vol. 27, no. 9, p. 110825, 2024.
- [6] Mikolajczyk, K., and C. Schmid, "A Performance Evaluation of Local Descriptors." *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 27, no. 10, 2005, pp. 1615–30.
- [7] "OpenCV: Feature Matching", [Online]. Available: https://docs.opencv.org/4.x/dc/dc3/tutorial_py_matcher.html. [Accessed: 9-Dec.-2025].
- [8] M. Yang et al., "Image Matching: Foundations, State of the Art, and Future Directions," *Journal of Imaging*, vol. 11, no. 10, p. 329, Sep. 2025